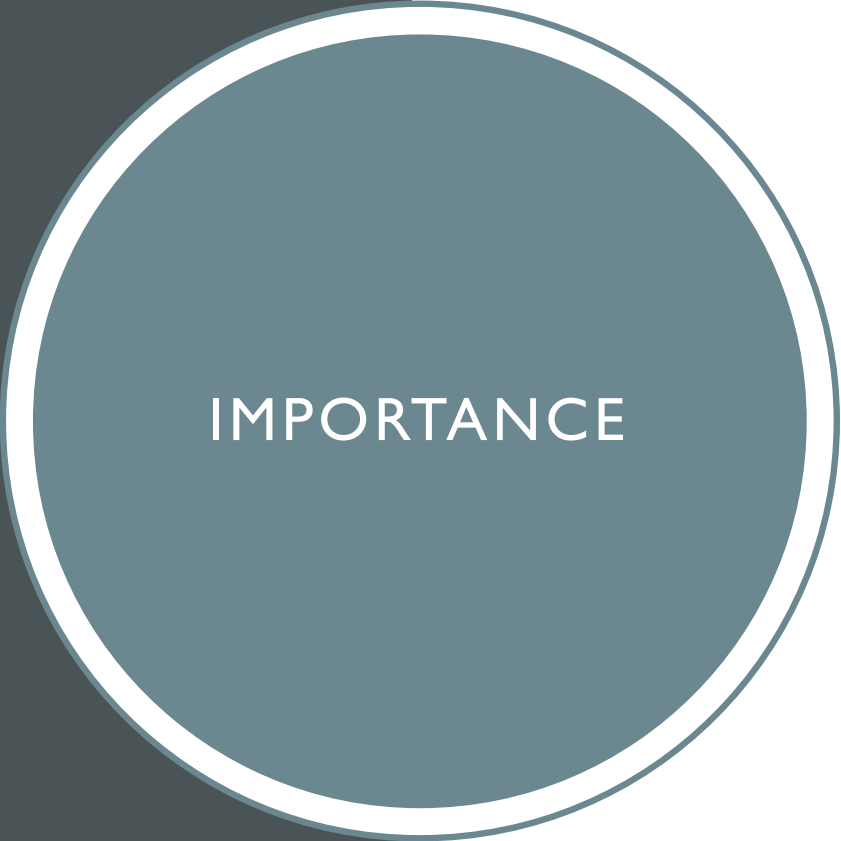


NEURAL BAG OF WORDS

THE VIBE CHECKER

Group 1: Popy Halder, Dayvon King, Nathaniel Osborn



IMPORTANCE

- The Neural Bag of Words model helps computers read text and sense emotions, spotting signs of distress or sadness even when people don't say it outright. By recognizing silent cries for help, NBow can help identifying those struggling with negative feelings.

METHODOLOGY

Setup: Load tools and fix random generators so results can be replicated

Data split : 25,000 review divided for training, validating and testing

Tokenization : words become numbers in a vocabulary dictionary; rare words are treated as special tokens and ignored (<unk>)

Batching : reviews are padded to fit in a matrix.

Neural network layer: embedding (assign meaning to numbers), Linear (sentimental judge), Mean pooling (average sentence meaning)

Training loop: model practice for 10 epochs of guessing, checking and correcting errors against labels

Learning process: uses CrossEntropyLoss to compare guess to real label (0 or 1), optimizer fine-tunes to improve accuracy

SETUP

```
# =====  
# 0. Configuration & Seeding  
# =====  
seed = 1234  
np.random.seed(seed)  
torch.manual_seed(seed)  
if torch.cuda.is_available():  
    torch.cuda.manual_seed(seed)  
    torch.backends.cudnn.deterministic = True  
  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print(f"Using device: {device}")
```

DATA SPLIT

```
print("Loading IMDb dataset from Hugging Face...")
# Load the dataset
dataset = datasets.load_dataset("imdb")
train_data = dataset["train"]
test_data = dataset["test"]

# Create validation split (25% of train)
train_valid_split = train_data.train_test_split(test_size=0.25)
train_data = train_valid_split["train"]
valid_data = train_valid_split["test"]

print(f"Train size: {len(train_data)}")
print(f"Valid size: {len(valid_data)}")
print(f"Test size: {len(test_data)}")
```

TOKENIZATION

```
# Robust English tokenizer replacement (lowercase + split)
def simple_tokenizer(text):
    # Add Robustness by removing punctuation using regex, then lowercase and split
    text = re.sub(f'[{re.escape(string.punctuation)}]', '', text)
    return text.lower().split()

# 1. Tokenize the training data
print("Tokenizing training data...")
def tokenize_batch(examples):
    return {"tokens": [simple_tokenizer(text) for text in examples["text"]]}

# We map this over the dataset.
# Note: We don't truncate to max_length here yet, we handle that in collation/padding
train_data = train_data.map(tokenize_batch, batched=True)
valid_data = valid_data.map(tokenize_batch, batched=True)
test_data = test_data.map(tokenize_batch, batched=True)

# 2. Build Vocabulary manually using Counter
print("Building vocabulary...")
token_counts = Counter()
for tokens in train_data["tokens"]:
    token_counts.update(tokens)

# Define special tokens
unk_token = "<unk>"
pad_token = "<pad>"
min_freq = 5

# Create vocab mapping: Token -> ID
# Only keep words that appear at least 5 times
sorted_vocab = sorted([token for token, count in token_counts.items() if count >= min_freq])
vocab = {token: i+2 for i, token in enumerate(sorted_vocab)} # +2 because 0 and 1 are reserved
vocab[unk_token] = 0
vocab[pad_token] = 1
```

BATCHING

```
# This function converts a list of tokens to a list of IDs
def text_pipeline(tokens):
    return [vocab.get(token, vocab[unk_token]) for token in tokens]

def collate_batch(batch):
    label_list, text_list = [], []

    for _item in batch:
        label_list.append(_item['label'])
        # Process text to IDs
        processed_text = torch.tensor(text_pipeline(_item['tokens']), dtype=torch.int64)
        # Truncate if necessary (optional, e.g., keep only first 256 words)
        if processed_text.size(0) > 256:
            processed_text = processed_text[:256]
        text_list.append(processed_text)

    # Pad sequences
    # vocab[pad_token] is 1
    text_list = nn.utils.rnn.pad_sequence(text_list, batch_first=True, padding_value=vocab[pad_token])
    label_list = torch.tensor(label_list, dtype=torch.int64)

    return {"ids": text_list, "label": label_list}

BATCH_SIZE = 512

# Create DataLoaders
train_loader = torch.utils.data.DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True, collate_fn=collate_batch)
valid_loader = torch.utils.data.DataLoader(valid_data, batch_size=BATCH_SIZE, shuffle=False, collate_fn=collate_batch)
test_loader = torch.utils.data.DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False, collate_fn=collate_batch)
```

NEURAL NETWORK LAYER

```
class NBow(nn.Module):
    def __init__(self, vocab_size, embedding_dim, output_dim, pad_index):
        super().__init__()
        # Embedding Layer
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=pad_index)
        # Linear Layer
        self.fc = nn.Linear(embedding_dim, output_dim)

    def forward(self, ids):
        # ids = [batch size, seq len]
        embedded = self.embedding(ids)
        # embedded = [batch size, seq len, embedding dim]

        # Mean Pooling (Bag of Words effect)
        # We need to be careful not to average the padding tokens
        # But for a simple primitive NBow, standard mean() is usually acceptable for tutorials
        pooled = embedded.mean(dim=1)

        prediction = self.fc(pooled)
        return prediction

# Initialize Model
vocab_size = len(vocab)
embedding_dim = 300
output_dim = 2
pad_index = vocab[pad_token]

model = NBow(vocab_size, embedding_dim, output_dim, pad_index)
model = model.to(device)

print(f"Model Parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad):,}")
```


TRAINING LOOP

```
optimizer = optim.Adam(model.parameters())
criterion = nn.CrossEntropyLoss().to(device)

def get_accuracy(prediction, label):
    batch_size, _ = prediction.shape
    predicted_classes = prediction.argmax(dim=-1)
    correct_predictions = predicted_classes.eq(label).sum()
    return correct_predictions / batch_size

def train_one_epoch(loader, model, criterion, optimizer):
    model.train()
    losses = []
    accs = []
    for batch in tqdm.tqdm(loader, desc="Training"):
        ids = batch["ids"].to(device)
        label = batch["label"].to(device)

        optimizer.zero_grad()
        prediction = model(ids)
        loss = criterion(prediction, label)
        acc = get_accuracy(prediction, label)

        loss.backward()
        optimizer.step()

        losses.append(loss.item())
        accs.append(acc.item())
    return np.mean(losses), np.mean(accs)

def evaluate(loader, model, criterion):
    model.eval()
    losses = []
    accs = []
    with torch.no_grad():
        for batch in loader:
            ids = batch["ids"].to(device)
            label = batch["label"].to(device)
            prediction = model(ids)
            loss = criterion(prediction, label)
            acc = get_accuracy(prediction, label)
            losses.append(loss.item())
            accs.append(acc.item())
    return np.mean(losses), np.mean(accs)

print("\nStarting Training...")
for epoch in range(10):
    train_loss, train_acc = train_one_epoch(train_loader, model, criterion, optimizer)
    valid_loss, valid_acc = evaluate(valid_loader, model, criterion)

    print(f"Epoch {epoch+1}: Train Loss {train_loss:.3f} (Acc: {train_acc:.3f}) | Valid Loss {valid_loss:.3f} (Acc: {valid_acc:.3f})")
```

INFERENCE

```
def predict_sentiment(text):
    model.eval()
    with torch.no_grad():
        tokens = simple_tokenizer(text)
        ids = [vocab.get(t, vocab[unk_token]) for t in tokens]
        tensor = torch.LongTensor(ids).unsqueeze(0).to(device)
        prediction = model(tensor)
        prob = torch.softmax(prediction, dim=-1)
        cls = prediction.argmax(dim=-1).item()
        return "POSITIVE" if cls == 1 else "NEGATIVE", prob[0][cls].item()

print("\n--- Testing ---")
print("This movie was terrible:", predict_sentiment("This movie was terrible"))
print("I loved this film:", predict_sentiment("I loved this film"))
```



CHALLENGES: TRUSTWORTHINESS

- Reviews or posts may be **biased**, sarcastic, or misleading
- Ignoring word order can **misinterpreted** words/ phrases
- People often express distress **indirectly**, through jokes or cultural references, which the model may miss
- **Mislabeled emotions** in sensitive areas like depression can cause harm, so trustworthiness is essential

MODEL SOPHISTICATION

Our model is primitive, it doesn't understand word order or sentence structure.

For example, "Good acting, bad plot" and "Bad acting, good plot" use the same words.

Because the model averages word meanings, it sees both sentences as identical.

Instead of grasping the full context, it just picks up on the overall vibe of the vocabulary.

This limits its ability to detect nuance, sarcasm, or emotional shifts in language.

CONCLUSION

- The model processed 25,000 reviews with speed and precision.
- It split the data into learning, practice, and testing sets, just like a student preparing for an exam.
- This helped build a vocabulary of over 38,000 words, teaching the model to understand everyday language.
- We included robustness in the tokenization to maintain its performance and accuracy under a wide range of conditions, including unexpected inputs, noisy data, or malicious attacks
- The model is primitive, but thoroughly transparent in its explainability and this allows for an exchange of trustworthiness

CITATION

- 1. Trevett, B. (2019, November). Neural Bag of Words [GitHub repository]. GitHub. <https://github.com/bentrevett/pytorch-sentiment-analysis/blob/main/1%20-%20Neural%20Bag%20of%20Words.ipynb>
- 2. dair-ai. (2020). Emotion Dataset [Dataset]. Hugging Face. <https://huggingface.co/datasets/dair-ai/emotion>

CONTRIBUTIONS

Nate: 34%: Code

Popy: 33%: PowerPoint presentation

Dayvon: 33%: Project Report creation

The background of the slide is a dense field of 3D dollar signs (\$). Most are dark grey, but one is a bright yellow lightbulb, positioned centrally behind the text box. Below the text box, there are a few orange 3D cubes. The text 'QUESTIONS?' is centered in a white box with a thin black border.

QUESTIONS?